

Football Tournament Management System

Developer Manual

Technical Documentation for Developers

Author: Rostand Cedrix Assonfo Demaboi
Institution: Technische Hochschule Georg Agricola, Bochum
Module: Introduction to Database Management Systems
Semester: Summer Term 2026

Version 1.0

September 10, 2026

Contents

1 Introduction

1.1 Purpose of this Document

This document is the technical developer manual for the Football Tournament Management System.

Its purpose is to provide future developers with the information required to understand, install, maintain, test, and extend the application.

Unlike the user manual, which focuses on how to use the application, this document focuses on the internal architecture, source code, database structure, API, business logic, and development workflow.

1.2 System Overview

The Football Tournament Management System is an application designed to organize, track, and administer football competitions.

The system supports the management of:

- tournaments,
- teams,
- matches,
- match scores,
- referees,
- tournament standings.

The application follows a decoupled architecture composed of a backend REST API, a graphical frontend, and a relational database.

1.3 User Roles

The system provides two main user roles.

1.3.1 Organizer Mode

Organizer Mode provides write access to the system.

An authenticated organizer can:

- create tournaments,
- modify tournaments,
- delete tournaments,
- create teams,
- modify teams,
- delete teams,

- schedule matches,
- update match scores,
- delete matches.

Authentication is performed using an API key transmitted through the **X-API-Key** HTTP header.

1.3.2 Viewer Mode

Viewer Mode is a read-only mode.

A viewer can:

- view tournaments,
- view teams,
- view matches,
- view match scores,
- view tournament standings.

2 Technology Stack

Component	Technology
Backend	Python / FastAPI
Frontend	Python / Tkinter
Database	Relational SQL Database
Communication	REST API
Authentication	X-API-Key
Containerization	Docker / Docker Compose

Table 1: Technology stack

2.1 Backend

The backend is implemented as a REST API using FastAPI.

Its responsibilities include:

- receiving HTTP requests,
- validating input data,
- applying business rules,
- communicating with the database,
- returning HTTP responses.

2.2 Frontend

The frontend is implemented using Python Tkinter.

It provides a graphical interface for:

- viewing tournaments,
- viewing teams,
- viewing matches,
- viewing standings,
- performing administrative operations.

2.3 Docker

Docker Compose is used to orchestrate the application services.

The goal is to provide a reproducible and portable runtime environment.

3 System Architecture

3.1 Architecture Overview

The application follows a client-server architecture.

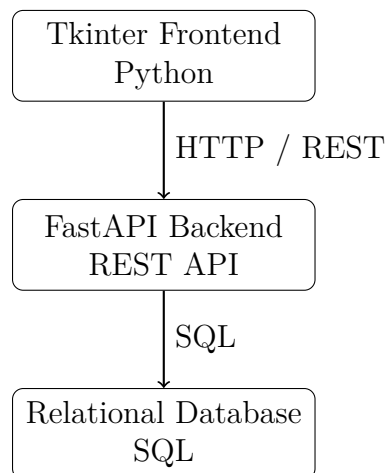


Figure 1: System architecture

3.2 Frontend Layer

The frontend is responsible for user interaction.

It communicates with the backend exclusively through the REST API.

The frontend should not directly access the database.

3.3 Backend Layer

The backend acts as the central application layer.

It is responsible for:

- API endpoints,
- authentication,
- validation,
- business logic,
- database access.

3.4 Database Layer

The database stores persistent application data.

The database is responsible for maintaining:

- primary key constraints,
- foreign key constraints,
- uniqueness constraints,
- referential integrity.

4 Project Structure

The following structure represents the recommended organization of the project.

```
1 football-tournament-manager/  
2 |  
3 +-- backend/  
4 |   +-- ...  
5 |  
6 +-- frontend/  
7 |   +-- football_tournament/  
8 |       +-- ...  
9 |  
10 +-- database/  
11 |   +-- ...  
12 |  
13 +-- tests/  
14 |   +-- ...  
15 |  
16 +-- docker-compose.yml  
17 +-- Dockerfile  
18 +-- requirements.txt  
19 +-- README.md  
20 +-- DEVELOPER_MANUAL.tex
```

4.1 Backend Directory

The backend directory contains the FastAPI application and all backend-related components.

TODO: Document the actual backend directories and files according to the source code.

4.2 Frontend Directory

The frontend directory contains the Tkinter graphical application.

TODO: Document the actual frontend modules and their responsibilities.

4.3 Tests Directory

The tests directory contains automated tests.

TODO: Document the actual test structure.

5 Installation and Setup

5.1 Prerequisites

The following software is required:

- Git,
- Docker,
- Docker Compose,
- Python 3,
- a SQL-compatible database environment.

TODO: Specify the exact supported versions.

5.2 Clone the Repository

```
1 git clone <repository-url>
2 cd football-tournament-manager
```

5.3 Backend Installation

TODO: Document the exact Python environment and dependency installation process.

Example:

```
1 python3 -m venv venv
2 source venv/bin/activate
3 pip install -r requirements.txt
```

5.4 Frontend Installation

The frontend uses a Python virtual environment.

Example:

```
1 cd frontend
2 source venv/bin/activate
```

6 Docker and Deployment

6.1 Starting the Services

The complete backend environment can be started using:

```
1 docker compose up --build -d
```

6.2 Checking Running Containers

```
1 docker compose ps
```

6.3 Viewing Logs

```
1 docker compose logs
```

To inspect a specific service:

```
1 docker compose logs <service-name>
```

6.4 Stopping the Services

```
1 docker compose down
```

TODO: Document the exact Docker services defined in `docker-compose.yml`.

7 Database Design

7.1 Database Overview

The application uses a relational database to persist tournament, team, and match information[cite: 1, 2].

7.2 Entity Relationship Model

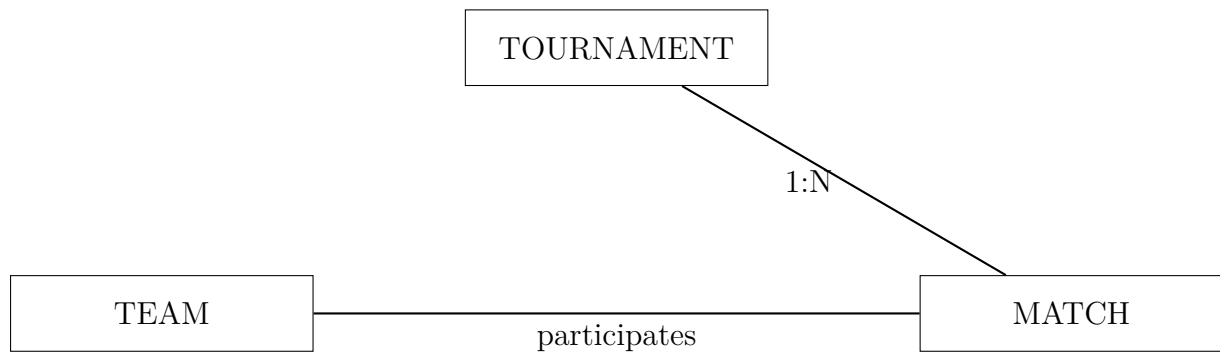


Figure 2: Simplified entity relationship model

TODO: Replace this simplified diagram with the exact database schema.

7.3 Tournament Table

Column	Type	Constraint	Description
id	—	Primary Key	Unique tournament identifier
name	—	—	Tournament name
start_date	—	—	Tournament start date
end_date	—	—	Tournament end date
status	—	—	Tournament status

TODO: Replace types and fields with the actual SQL schema.

7.4 Team Table

Column	Type	Constraint	Description
id	—	Primary Key	Unique team identifier
name	—	—	Official team name
coach	—	—	Coach or manager

7.5 Team/Tournament Association

The many-to-many ($N : M$) relationship between teams and tournaments is implemented using the `team_tournaments` associative entity, ensuring that a team can participate in multiple tournaments and a tournament can include multiple teams[cite: 1, 2].

- **Table Name:** `team_tournaments` (or `TeamTournament`)[cite: 1, 2]
- **Composite Primary Key:** (`team_id`, `tournament_id`)[cite: 1, 2]
- **Foreign Keys:**

- `team_id` references `teams(team_id)`[cite: 1, 2]
- `tournament_id` references `tournaments(tournament_id)`[cite: 1, 2]
- **Business Purpose:** Prevents duplicate team registrations in the same tournament while maintaining clean referential integrity upon deletion[cite: 1, 2].

7.6 Match Table

Column	Type	Constraint	Description
<code>id</code>	—	Primary Key	Unique match identifier
<code>tournament_id</code>	—	Foreign Key	Related tournament
<code>home_team_id</code>	—	Foreign Key	Home team
<code>away_team_id</code>	—	Foreign Key	Away team
<code>date</code>	—	—	Match date
<code>home_score</code>	—	—	Home team score
<code>away_score</code>	—	—	Away team score
<code>referee</code>	—	—	Assigned referee

8 REST API

8.1 API Base URL

The default development API URL is:

```
1 http://localhost:8000
```

8.2 Authentication

Administrative operations require the HTTP header:

```
1 X-API-Key: <API-KEY>
```

Example:

```
1 curl -X POST http://localhost:8000/tournaments \
2     -H "X-API-Key: mykey"
```

8.3 Tournament Endpoints

Method	Endpoint	Description
GET	<code>/tournaments</code>	List tournaments
POST	<code>/tournaments</code>	Create tournament
PUT	<code>/tournaments/{id}</code>	Update tournament
DELETE	<code>/tournaments/{id}</code>	Delete tournament

TODO: Replace these endpoints with the exact endpoints implemented by the back-end.

8.4 Team Endpoints

Method	Endpoint	Description
GET	/teams	List teams
POST	/teams	Create team
PUT	/teams/{id}	Update team
DELETE	/teams/{id}	Delete team

8.5 Match Endpoints

Method	Endpoint	Description
GET	/matches	List matches
POST	/matches	Create match
PUT	/matches/{id}	Update match
DELETE	/matches/{id}	Delete match

9 Backend Architecture

9.1 Request Processing

A typical API request follows this sequence:

1. The frontend sends an HTTP request.
2. FastAPI receives the request.
3. Authentication is checked when required.
4. Input data is validated.
5. Business logic is executed.
6. The database is queried or modified.
7. The backend returns an HTTP response.
8. The frontend updates its display.

9.2 API Routers

TODO: Document the actual FastAPI routers implemented in the source code.

Example:

```
1 @router.get("/tournaments")
2 def get_tournaments():
3     ...
```

9.3 Data Validation

TODO: Document the Pydantic schemas and validation rules.

10 Frontend Architecture

10.1 Main Interface

The graphical interface is organized around several tabs:

- Tournaments,
- Teams,
- Matches,
- Standings.

10.2 Connection Panel

At startup, the frontend requests:

- API URL,
- optional X-API-Key.

Leaving the API key empty provides Viewer Mode.

Providing a valid administrative key enables Organizer Mode.

10.3 Tournament Tab

The Tournament tab displays the registered competitions.

10.4 Teams Tab

The Teams tab displays participating clubs and their information.

10.5 Matches Tab

The Matches tab provides access to fixtures and scores.

A tournament selector allows filtering matches by tournament.

10.6 Standings Tab

The Standings tab dynamically calculates tournament rankings.

11 Business Logic

11.1 Match Management

A match is associated with:

- one tournament,
- one home team,
- one away team,
- a date,
- optionally a score,
- a referee.

11.2 Score Management

Scores can be updated by an authenticated organizer.

11.3 Standings Calculation

The standings are calculated from the results of matches.

The ranking contains at least:

- points,
- wins,
- draws,
- losses,
- goals scored,
- goals conceded,
- goal difference.

TODO: Document the exact ranking and tie-breaking algorithm.

For example:

$$\text{Goal Difference} = \text{Goals For} - \text{Goals Against}$$

TODO: Specify the exact points system implemented by the application.

12 Authentication and Authorization

12.1 API Key

The API uses an API key mechanism based on the HTTP `X-API-Key` header.

12.2 Viewer Permissions

Viewer users have read-only access.

12.3 Organizer Permissions

Organizer users have read and write access.

Operation	Viewer	Organizer
Read tournaments	Yes	Yes
Read teams	Yes	Yes
Read matches	Yes	Yes
Read standings	Yes	Yes
Create data	No	Yes
Update data	No	Yes
Delete data	No	Yes

Table 8: Authorization matrix

13 Error Handling

The API should provide meaningful HTTP status codes.

Status	Meaning
200	Successful request
201	Resource successfully created
400	Invalid request
401	Unauthorized
404	Resource not found
409	Database or resource conflict
500	Internal server error

Table 9: HTTP error codes

TODO: Verify the exact status codes implemented by the API.

14 Database Integrity

Database operations must preserve referential integrity.

Examples include:

- a match must reference an existing tournament,
- a match must reference existing teams,
- referenced entities should not be deleted if this would violate database constraints.

The application respects database referential integrity constraints when deleting obsolete teams or matches.

TODO: Document the exact foreign-key and ON DELETE rules.

15 Testing

15.1 Testing Strategy

The project should test the following layers:

- database,
- business logic,
- REST API,
- frontend.

15.2 Unit Tests

Unit tests should validate isolated business rules.

Example:

```
1 def test_goal_difference():
2     assert calculate_goal_difference(3, 1) == 2
```

15.3 API Tests

API tests should verify:

- successful requests,
- invalid requests,
- authentication,
- missing resources,
- database conflicts.

15.4 Database Tests

Database tests should verify:

- primary keys,
- foreign keys,
- uniqueness,
- deletion constraints.

16 Development Workflow

A recommended development workflow is:

1. Pull the latest version of the repository.
2. Create a dedicated feature branch.
3. Implement the change.
4. Run automated tests.
5. Test the API.
6. Test the frontend.
7. Verify database integrity.
8. Update the documentation.
9. Commit the changes.
10. Create a pull request.

Example:

```
1 git checkout -b feature/new-functionality
2
3 # Implement changes
4
5 git add .
6 git commit -m "Add new functionality"
7
8 git push origin feature/new-functionality
```

17 Troubleshooting

17.1 Backend Does Not Start

Check the Docker services:

```
1 docker compose ps
```

Then inspect the logs:

```
1 docker compose logs
```

17.2 Frontend Cannot Connect

Verify that the API is running on:

```
1 http://localhost:8000
```

Also verify the API URL entered in the connection dialog.

17.3 Organizer Mode Does Not Work

Check that the correct API key is provided through the **X-API-Key** field.

TODO: Document how the API key is configured in the actual application.

18 Maintenance

18.1 Dependency Updates

Document how Python dependencies are updated.

Example:

```
1 pip install --upgrade <package>
2 pip freeze > requirements.txt
```

18.2 Database Changes

Database schema changes should be documented and tested before being deployed.

TODO: Document whether the project uses SQL migration tools such as Alembic or another migration mechanism.

18.3 Logging

TODO: Document the application's logging system and log locations.

19 Extending the Application

Future developers may extend the application with additional functionality.

Potential extensions include:

- player management,
- yellow and red cards,
- stadium management,
- player statistics,
- tournament groups,
- notifications,
- additional competition formats.

19.1 Adding a New Feature

A typical feature implementation should follow:

1. Modify the database schema if necessary.
2. Update the database model.
3. Add or modify business logic.
4. Add REST API endpoints.
5. Add frontend functionality.
6. Add automated tests.
7. Update the developer documentation.

20 Deployment

20.1 Development Environment

The development environment uses Docker Compose and a local frontend environment.

20.2 Production Environment

TODO: Document the actual production deployment procedure if a production environment exists.

Important elements to document include:

- server configuration,
- database configuration,
- environment variables,
- API configuration,
- network ports,
- security configuration,
- backup strategy.

21 Appendix

21.1 Useful Commands

Command	Purpose
<code>docker compose up --build -d</code>	Start services

<code>docker compose ps</code>	Show running services
<code>docker compose logs</code>	Show logs
<code>docker compose down</code>	Stop services

21.2 Default Configuration

Parameter	Value
API URL	<code>http://localhost:8000</code>
Authentication Header	<code>X-API-Key</code>

21.3 Documentation Status

This document contains sections marked with **TODO**.

These sections must be completed using the actual source code, database schema, API implementation, Docker configuration, and tests before the document is considered complete.